Interview Prep: Intermediate ML Engineer & Intermediate Python

> Use AI-generated study material as a starting point — fact-check answers and adapt them to your own experience before relying on them in an interview.

Part 1 — Intermediate ML Engineer (13 Questions)

1. Explain the bias-variance tradeoff.

- **Bias** is error from overly simplistic assumptions; high bias underfits (misses real patterns).
- **Variance** is error from sensitivity to training data; high variance overfits (memorizes noise).
- The goal is to balance the two to minimize total generalization error. Increasing model complexity lowers bias but raises variance, and vice versa.

2. How do you handle an imbalanced dataset?

- **Resampling:** oversample the minority class (e.g., SMOTE) or undersample the majority class.
- **Class weights:** penalize misclassifying the minority class more heavily in the loss function.
- **Better metrics:** use precision, recall, F1, or PR-AUC instead of raw accuracy.
- **Threshold tuning:** adjust the decision threshold rather than defaulting to 0.5.

3. What's the difference between bagging and boosting?

- **Bagging** (e.g., Random Forest) trains models in parallel on bootstrapped samples and averages them; it reduces variance.
- **Boosting** (e.g., XGBoost, AdaBoost) trains models sequentially, each correcting the previous one's errors; it reduces bias.
- Bagging is more robust to overfitting; boosting often achieves higher accuracy but is more sensitive to noise and tuning.

4. Compare L1 and L2 regularization.

- **L1 (Lasso)** adds the absolute value of weights to the loss; it drives some weights to exactly zero, performing feature selection.
- **L2 (Ridge)** adds the squared value of weights; it shrinks weights smoothly toward zero but rarely to exactly zero.

- L1 produces sparse models; L2 handles correlated features more gracefully. Elastic Net combines both.

5. What is cross-validation and why use it?

- It splits data into k folds, trains on k−1 and validates on the remaining fold, rotating through all folds.
- It gives a more reliable estimate of generalization performance than a single train/test split, and uses data more efficiently.
- Use **stratified** k-fold for classification to preserve class proportions, and time-series split for temporal data.

6. When is feature scaling necessary?

- **Needed** for distance-based and gradient-based algorithms: KNN, SVM, K-means, PCA, linear/logistic regression, and neural networks.
- **Not needed** for tree-based models (decision trees, random forests, gradient boosting), which split on thresholds.
- Common methods: standardization (zero mean, unit variance) and min-max normalization (scale to [0,1]).

7. Explain precision, recall, and F1 — when do you favor each?

- **Precision** = TP / (TP + FP): of predicted positives, how many were correct. Favor it when false positives are costly (e.g., spam filters).
- **Recall** = TP / (TP + FN): of actual positives, how many were caught. Favor it when false negatives are costly (e.g., cancer screening).
- **F1** is the harmonic mean of the two — useful when you need a single balanced metric on imbalanced data.

8. What are the main variants of gradient descent?

- **Batch GD:** uses the entire dataset per step — stable but slow and memory-heavy.
- **Stochastic GD (SGD):** one sample per step — fast and noisy, can escape local minima.
- **Mini-batch GD:** a small batch per step — the practical default, balancing speed and stability.
- Optimizers like Momentum, RMSProp, and Adam build on these to adapt the step size.

9. How do you detect and prevent overfitting?

- **Detect:** a large gap between training and validation performance, or validation error rising while training error falls.
- **Prevent:** more data, regularization (L1/L2, dropout), early stopping, simpler models, cross-validation, and data augmentation.

10. How do you handle missing data?

- **Remove** rows/columns when missingness is small or the feature is mostly empty.
- **Impute** with mean/median/mode, model-based methods (KNN, regression), or a dedicated "missing" category.
- **Flag** missingness with an indicator column, since the fact that a value is missing can itself be informative.
- Always reason about *why* data is missing (random vs. systematic) before choosing a strategy.

11. What's the difference between generative and discriminative models?

- **Discriminative models** learn the decision boundary P(y|x) directly (e.g., logistic regression, SVM).
- **Generative models** learn the joint distribution P(x,y) and can generate new samples (e.g., Naive Bayes, GANs, VAEs).
- Discriminative models usually win on pure classification accuracy; generative models are needed when you want to model or sample the data distribution.

12. How do you monitor a model in production?

- Track **data drift** (input distribution shifts) and **concept drift** (the input–output relationship changes).
- Monitor live performance metrics, latency, and prediction distributions against a baseline.
- Set up alerts and a retraining trigger/schedule, and keep model versioning for rollback.

13. What approaches do you use for hyperparameter tuning?

- **Grid search:** exhaustive over a defined grid — thorough but expensive.
- **Random search:** samples randomly — more efficient in high-dimensional spaces.
- **Bayesian optimization** (e.g., Optuna, Hyperopt): uses past results to choose promising next trials.
- Always tune against a validation set or via cross-validation, never on the test set.

Part 2 — Intermediate Python (20 Questions)

1. List vs. tuple — what's the difference?

- Lists are **mutable** (`[]`); tuples are **immutable** (`()`).

- Tuples are slightly faster, can be used as dictionary keys, and signal "this shouldn't change." Lists are for collections you'll modify.

2. Explain mutable vs. immutable types.

- **Immutable:** int, float, str, tuple, frozenset, bool — cannot be changed after creation.
- **Mutable:** list, dict, set, and most custom objects — can be changed in place.
- This affects how arguments behave when passed to functions and how objects are shared.

3. What is a list comprehension?

- A concise way to build a list: `[x**2 for x in range(10) if x % 2 == 0]`.
- It's more readable and usually faster than an equivalent `for` loop with `.append()`. Dict and set comprehensions follow the same syntax with `{}`.

4. What are generators and the `yield` keyword?

- A generator produces values lazily, one at a time, instead of building the whole sequence in memory.
- `yield` pauses the function and resumes on the next call. Great for large or infinite sequences:

```python
def count_up(n):
    i = 0
    while i < n:
        yield i
        i += 1
```

5. What is a decorator?

- A function that takes another function and extends its behavior without modifying it, applied with `@`.

```python
```

```python
def log(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log
def greet(): ...
```

- Common for logging, timing, authentication, and caching.

6. What do `*args` and `**kwargs` do?

- `*args` collects extra **positional** arguments into a tuple.
- `**kwargs` collects extra **keyword** arguments into a dict.
- They let functions accept a variable number of arguments.

7. Shallow copy vs. deep copy?

- A **shallow copy** (`copy.copy`) copies the outer object but shares references to nested objects.
- A **deep copy** (`copy.deepcopy`) recursively copies everything, so there's no shared reference.
- Matters most with nested mutable structures like lists of lists.

8. What is the GIL?

- The **Global Interpreter Lock** allows only one thread to execute Python bytecode at a time in CPython.
- It limits CPU-bound multithreading. Use **multiprocessing** for CPU-bound work and threads for I/O-bound work. (Note: recent Python versions are introducing an optional no-GIL build.)

9. What is a context manager?

- An object that manages setup and cleanup via `with`, guaranteeing resources are released even if an error occurs:

```python
with open("file.txt") as f:
    data = f.read()
```

- Implemented with `__enter__`/`__exit__` or the `@contextmanager` decorator.

10. What is a lambda function?

- An anonymous, single-expression function: `square = lambda x: x**2`.
- Useful for short throwaway functions, often as the `key` argument in `sorted`, `map`, or `filter`. Use a named `def` for anything complex.

11. Difference between `is` and `==`?

- `==` compares **values** (equality).
- `is` compares **identity** (whether two names point to the same object in memory).
- Use `is` only for singletons like `None`: `if x is None:`.

12. How does exception handling work?

- `try` runs risky code, `except` catches specific errors, `else` runs if no error, `finally` always runs (for cleanup).

```python
try:
    result = 10 / x
except ZeroDivisionError:
    result = None
finally:
    print("done")
```

- Catch specific exceptions rather than a bare `except`.

13. `dict[key]` vs. `dict.get(key)`?

- `dict[key]` raises `KeyError` if the key is missing.
- `dict.get(key)` returns `None` (or a supplied default) instead of raising.
- Use `.get()` when a missing key is expected and you want a fallback.

14. What do `enumerate()` and `zip()` do?

- `enumerate(iterable)` yields `(index, value)` pairs — cleaner than tracking a counter manually.
- `zip(a, b)` pairs elements from multiple iterables into tuples, stopping at the shortest one.

15. What's the mutable default argument pitfall?

- A default like `def f(x, items=[])` is created **once** and shared across calls, so it accumulates state unexpectedly.

- Fix it with `None`:

```python
def f(x, items=None):
    if items is None:
        items = []
```

16. `@staticmethod` vs. `@classmethod`?

- `@staticmethod` receives no implicit first argument — it's just a function namespaced inside a class.
- `@classmethod` receives the class as `cls` and can access/modify class state; often used for alternative constructors.

17. What does slicing do, and what is `[::-1]`?

- Slicing extracts a subsequence: `seq[start:stop:step]`.
- `seq[::-1]` reverses a sequence by stepping backward. Slicing returns a new object and doesn't mutate the original.

18. When would you use a set?

- For storing **unique** elements and fast membership tests (`O(1)` average lookup).
- For set operations: union (`|`), intersection (`&`), difference (`-`). Sets are unordered and hold only hashable items.

19. Explain `map`, `filter`, and `reduce`.

- `map(func, iterable)` applies a function to every element.
- `filter(func, iterable)` keeps elements where the function returns `True`.
- `reduce(func, iterable)` (from `functools`) cumulatively combines elements into one value.
- Comprehensions are often preferred over `map`/`filter` for readability.

20. `global` vs. `nonlocal`?

- `global` lets a function rebind a variable at module (global) scope.
- `nonlocal` lets a nested function rebind a variable in the nearest enclosing (but non-global) scope.
- Both are needed only when **reassigning** outer variables; reading them doesn't require either keyword.

Reminder: these are model answers meant to guide your preparation — verify technical details and frame responses around your own projects and experience.